Sahayak testing report

Prototype for Ideas for India 2026 · generated 2026-10-07 by `scripts/build_testing_report.py` from the result files in `bench/results/`. Every number below can be reproduced with the command given in its section. The benchmarks of 2–6 Oct ran on a 4-core laptop CPU (Intel i5-10210U, no GPU) with the internet off; the outside tests of 7 Oct (real messages, blind red team, the language-model baseline, the stress test) ran on an Apple M3 Mac, and each of those sections says so.

At a glance

Area	Result	How it was measured
Scam detection	34 / 37 scams caught, recall 91.9% (95% Wilson CI 78.7%–97.2%); false alarms 2 / 23 genuine (2.4%–26.8%); keyword blocklist: recall 51.4%, false alarms 43.5%	ScamBench v0 frozen test split, 60 messages written by the team
Speed of a verdict	2.2 ms median, 3.0 ms 95th percentile	same run
Scheme rules	2,400 / 2,400 decisions match a second transcription of the rules by the same author	SchemeBench v1, 200 personas × 12 schemes
Interview length	median 5 questions, never more than 7 (limit 8); skipping questions changed 0 of 224,640 answer combinations	SchemeBench v1, exhaustive check
Hindi speech recognition	word error rate 15.4% (95% CI 14.0%–16.8%)	Google FLEURS Hindi test, 418 recordings, 81 min of adults reading sentences aloud
Voice reply time	0.65 s median from releasing the mic to the reply's first audio (2.44 s when the reply is synthesised live)	`bench/voice_latency.py`
Screenshot reading	same scam / not-scam decision as the typed message on 51 / 55 screenshots; 6.86 s each	ScamBench test messages rendered as SMS screenshots
Red team	disguised scams flagged 46 / 52 on the first run (blocklist 33 / 52); hard genuine messages left alone 19 / 22	74 messages written to get past Sahayak
Real published messages	67 / 97 (69%) scams caught (blocklist 62 / 97 (64%)), 11 / 34 (32%) genuine flagged (blocklist 17 / 34 (50%)), Hindi / English / Hinglish, first run	PublicBench v0: 137 messages published by the Income Tax portal, PIB Fact Check, courts and fact-checkers
Blind red team	Hindi / English / Hinglish: 92 / 103 (89%) scams caught (blocklist 51 / 103 (50%)), 14 / 59 (24%) false alarms on hard genuine messages; other Indian languages: "could not check", never a false green	182 messages by a separate AI model working blind, scored once
An AI model as the judge	qwen2.5:3b zero-shot on the real published messages: 99 / 103 (96%) scams caught but 21 / 34 (62%) genuine flagged, 2.13 s a message on a GPU; Sahayak 67 / 103 (65%) and 11 / 34 (32%), 1.7 ms	`bench/eval_llm_baseline.py`
Under load	30 phones at once: a check in 66 ms median; 0 errors in 5,842 requests; 360 hostile inputs, 0 unexpected answers	`bench/stress/run_stress.py`
Call descriptions	18 / 18 scam calls flagged, 8 / 8 genuine calls left alone	26 descriptions written by the team (a tuning set)

Area	Result	How it was measured
On the phone, offline	the phone's own engines give the node's exact answer on 1,841 scam checks and 7,983 benefits cases (0 mismatches); a check takes 0.154 ms	`bench/eval_phone_parity.py`, `scripts/check_phone_offline.py`
Zero egress	Sahayak's own process opens no outbound connection and makes no outside DNS lookup (counted by an audit hook on every socket); the firewall scripts block everything but the local network	`/app/node.html`, `tests/test_node.py`
Privacy of the impact export	messages full of names, phone numbers and UPI IDs leave no trace in the export; counts under 5 are masked	`tests/test_console.py`
Signed content	an altered pack is refused; an unsigned one is refused in strict mode	`tests/test_node.py`
Automated tests	489 tests pass, 1 skipped	`python -m pytest`

1. Scam detection: ScamBench v0

Data. 60 messages in the frozen test split (37 scams, 23 genuine) in English, Hindi and Hinglish, from a 256-message set written by the team to mirror publicly reported scam patterns and real bank, government and personal messages. Splits are made by message family, so near-copies never sit in both training and test (leakage check: 0). See `bench/scambench/DATA_CARD.md`.

Method. Rules, packs and thresholds were frozen, then the test split was scored once. A message counts as flagged when the verdict is Scam or Suspicious. Intervals in the table are 95% bootstrap intervals (10,000 resamples), as frozen; with only 37 scams, a bootstrap interval can reach 100%, so the summary above gives the 95% Wilson interval instead. The baseline is a keyword blocklist of the kind many SMS filters use.

System	Recall	Precision	False alarms on genuine
Keyword blocklist	51.4% (35.1%–67.4%)	65.5%	43.5%
Sahayak (signals + patterns + classifier)	91.9% (82.1%–100.0%)	94.4% (85.7%–100.0%)	8.7% (0.0%–22.2%)

McNemar's exact test, Sahayak against the blocklist: p = 5.6e-06.

What this does not show. The same team wrote the rules and the messages, so these numbers are optimistic. Errors found on the test split were fixed afterwards and logged (`bench/results/scambench_v0_test.md`, post-freeze log); the frozen numbers above stay the ones to quote. ScamBench v1 adds real messages collected with consent and is the honest test.

Without the sender. No ScamBench scam carries a sender, while most genuine messages carry a bank-style header, so a reviewer asked what happens when a person pastes only the text. Checked today with the installed packs: 0 of 105 genuine ScamBench messages are flagged with their sender, 2 of 105 without it.

Reproduce: `python bench/build_scambench.py`, `python bench/train_models.py`, `python bench/eval_scambench.py --split test --report`.

Red team v0

74 messages written to get past Sahayak: 52 known scams each disguised with one technique (hidden characters, spaced, look-alike or digit-for-letter spelling, links written in words or behind a short link, a warning wrapped around the ask, a spoofed bank sender, no link at all, newer scam types) and 22 genuine messages that share words with

scams. Scored once before any change: Sahayak flagged 46 / 52 disguised scams and left 19 / 22 genuine messages alone; the keyword blocklist 33 / 52 and 12 / 22.

The misses were then fixed (digits read as letters, written-out links undone, words people use instead of "OTP"); re-scored: 52 / 52 and 21 / 22, with ScamBench unchanged. Two genuine messages stay flagged by design: a shop offer on a short link, and a debit alert asking you to call or SMS a mobile number. The re-score is not an unbiased test; the first run is the number to quote.

Reproduce: `python bench/build_redteam.py`, `python bench/eval_redteam.py`; details in `bench/results/redteam_v0.md`.

Real published messages: PublicBench v0

The first set the team did not write: 137 messages people in India actually received (103 scams, 34 genuine), as published by the Income Tax portal's archive of its own SMS, PIB Fact Check, consumer-court orders that quote bank SMS, and fact-checkers and newspapers, 2023–2026; collected by an AI research agent that never saw the code, phone numbers replaced (`bench/public/DATA_CARD.md`). Scored once on 2026-10-07 (fraud pack 1.5.0), before anyone read an error.

	Scams caught (95% Wilson CI)	Genuine flagged (95% Wilson CI)
Sahayak, Hindi / English / Hinglish	67 / 97 (69%) (59.3%–77.4%)	11 / 34 (32%) (19.1%–49.2%)
Keyword blocklist, same messages	62 / 97 (64%)	17 / 34 (50%)
Sahayak, word-for-word messages only	53 / 75 (71%)	11 / 32 (34%)
Sahayak, other Indian languages	0 caught, 6 "could not check", 0 false greens	–

Real messages are harder than the team's own: the same engine caught 92% of ScamBench's test scams. What it missed on the first run: police and court-notice threats that ask for nothing yet, "letters of guarantee" from "RBI" asking for a tax, investment-group openers, "I sent you a message by mistake, forward it". What it flagged: genuine notices that use a scam's own words (an SBI maintenance notice, a prepaid-power warning, Income Tax "urgent" reminders).

Method for what came next. The set is split in two by a fixed hash of each id, made before anyone read an error. Fixes may learn from the dev half only; the test half's messages stay unread, so its score after the fixes is a fair estimate of the improvement.

After the fixes. 7 Oct, fraud pack 1.6.0: the blind red team's fixes, plus fixes learned from the DEV half only: fake police and court notices, a fee to 'release' a payment, prize letters, a relative 'arrested' and money to 'clear his name', Hindi blackmail, 'I accidentally sent you a message, forward it', a call asking you to dial the code you received, a QR code named before 'scan it and receive', stock-group openers, and a told call is never a bank alert. A rule that a message asking for nothing counts against a scam was tested and rejected: it lost 8 catches. 7 Oct, fraud pack 1.7.0: rules for the last live-demo traps, learned from the blind red team and the DEV half only: a call-forwarding code (*401*, **21*, also said aloud), an advance or token for something you cannot see first (keys by courier, no site visit, a ticket on WhatsApp), a fee through a link for a failed delivery, a link split with spaces ('remove the spaces'), a free-scheme post that says click on the photo, and 'confirmation code' read out on a call; fewer false alarms from brands' own short links and India Post's customs-duty notice. Measured on every set before adoption: no catch lost, no new false alarm. On the unread test half, scams caught 28 / 44 (64%) → 31 / 44 (70%), genuine flagged 6 / 20 (30%) → 6 / 20 (30%). All Hindi / English / Hinglish messages: 86 / 97 (89%) caught, 11 / 34 (32%) flagged (the dev half informed the fixes). The first run stays the number to quote.

Reproduce: `python bench/eval_public_v0.py --report`; every item's source is in `bench/public/public_messages_v0.provenance.tsv`.

Blind red team v1

182 messages written on 6–7 Oct 2026 by a separate AI model working blind: it never saw Sahayak's code, rules, packs or test sets, from 2025–26 advisories by I4C, police, banks and news reports: scams old and new (digital arrest,

"wrong number" investment openers, call forwarding by USSD code, eSIM swap, e-challan and wedding-invitation APKs, voter-roll OTPs) and deliberately hard genuine messages (the delivery OTP you do give at the door, the government's own "there is no digital arrest" message, bank alerts). 118 scams, 64 genuine; 20 of them in Indian languages Sahayak does not read yet. Scored once on 2026-10-07 (fraud pack 1.5.0, on the dev Mac), before anyone on the team read them.

	Scams caught (95% Wilson CI)	False alarms on genuine (95% Wilson CI)
Sahayak, Hindi / English / Hinglish	92 / 103 (89%) (81.9%–93.9%)	14 / 59 (24%) (14.7%–36.0%)
Keyword blocklist, same messages	51 / 103 (50%)	25 / 59 (42%)
Sahayak, other Indian languages	5 / 15 (33%) caught; 10 "could not check"; 0 given a false green	1 / 5 (20%)

On the first run Sahayak missed scams that arrive as a story with no link, number or code yet (a matrimonial match moving to "gold trading", a medical-charity appeal, a flat rented by an "officer" who cannot show it) and flagged genuine messages that look like scams on purpose (a delivery OTP given at the door, a gas-booking code, salary and debit alerts with an "SMS BLOCK" line, job-interview calls, family money requests). Every miss is listed in `bench/results/redteam_v1_blind.md`.

After the first run (post-freeze log in the results file): 7 Oct, fraud pack 1.6.0: fixes for the first run's errors (a bank's SMS-BLOCK footer beside its toll-free number, calm family requests, the writer's own bank, the delivery code given at the door, invitations to come in person or bring papers, power cuts for maintenance, discounts on bank cards, Hinglish and Hindi blackmail, gold-trading pitches, donation appeals to a personal UPI ID, job fees, money 'sent by mistake'). Each change was measured on every set before it was kept: 0 lost catches, 0 new false alarms. Two diagnosis groups (USSD call forwarding; spaced-out links and booking advances) did not finish and their misses remain. 7 Oct, fraud pack 1.7.0: a call-forwarding code (*401*, **21*, also said aloud: rt1-157), an advance or token for something you cannot see first (rt1-071 rent, rt1-120 helicopter seats, rt1-158 plot), a fee through a link for a failed delivery and a link split with spaces (rt1-026), and 'confirmation code' read out on a call (rt1-100). Each rule was also tested on new wordings and genuine look-alikes (tests/test_signals_1_7.py) and measured on every set: no catch lost, no new false alarm. Re-scored on fraud pack 1.7.0: 103 / 103 (100%) scams caught and 0 / 59 (0%) false alarms in Hindi / English / Hinglish. These messages shaped the fixes, so this is not an unbiased test; the first run stays the number to quote.

Reproduce: `python bench/redteam/build_redteam_v1.py`, `python bench/eval_redteam_v1.py --report`.

Why not just ask an AI model?

The first-round proposal had a large language model decide. The prototype measures that road: qwen2.5:3b (the largest model that ran at a usable speed on the team's laptop), zero-shot, temperature 0, one fixed prompt, on the same messages, through Ollama on Apple M3 (Darwin arm64) (its GPU; a CSC node's CPU would be several times slower).

Set	System	Scams caught	False alarms on genuine	Time per message
ScamBench v0 test split	qwen2.5:3b	36 / 37 (97%)	7 / 23 (30%)	median 1.64 s
ScamBench v0 test split	Sahayak	36 / 37 (97%)	0 / 23 (0%)	median 1.05 ms
Blind red team v1	qwen2.5:3b	108 / 118 (92%)	24 / 64 (38%)	median 2.24 s
Blind red team v1	Sahayak	97 / 118 (82%)	15 / 64 (23%)	median 1.76 ms
PublicBench v0 (real messages)	qwen2.5:3b	99 / 103 (96%)	21 / 34 (62%)	median 2.13 s
PublicBench v0 (real messages)	Sahayak	67 / 103 (65%)	11 / 34 (32%)	median 1.7 ms

Reproduce: `ollama pull qwen2.5:3b`, `python bench/eval_llm_baseline.py --report`; every answer and the model's one-line reason are in `bench/results/llm_baseline.json`.

---

2. Benefits Navigator: SchemeBench v1

Data. 200 personas (20 written as real-life stories, 180 generated around the ages where a rule changes) answering all seven interview questions. The expected answer for each of the 12 schemes comes from `bench/schemebench/oracle.py`, a separate transcription of the official rules as plain code that never reads the scheme pack.

Check	Result
Decisions matching the oracle	2,400 / 2,400
Questions per interview	median 5, maximum 7
Short interview reaches the same result as answering everything	200 / 200 personas; 224,640 / 224,640 answer combinations in the exhaustive check
Rupee amounts not found in the signed pack	0

What this does not show. The oracle and the pack share an author, so agreement rules out transcription slips, not a shared misreading of a rule. A second person deriving the answers from the official pages alone (`bench/schemebench/worksheet_v1.csv`) is the remaining acceptance step. Amounts are the central government's share; state top-ups are not modelled yet.

Reproduce: `python bench/build_schemebench.py`, `python bench/eval_schemebench.py --exhaustive --report`.

---

3. Voice

Hindi speech recognition. The node's own recognition code (Vosk small Hindi model, offline) on Google's FLEURS Hindi test split: 418 recordings of adults reading Wikipedia sentences aloud. Word error rate 15.4% (95% CI 14.0%–16.8%), at 0.232 × real time on the laptop CPU. The PRD target is at most 20% on VoiceBench. This is a public reference point, not a field result: long formal sentences are harder than the short answers people give Sahayak, but clear read speech by literate adults is easier than an older person speaking in a noisy shop. VoiceBench (the team's consented recordings from at least 10 speakers, three aged 55 or over, collected with `/app/voicebench.html`) is still to be recorded.

Reply time. From releasing the mic to the reply's first audio being ready (recognise and match the answer, pick the next question, synthesise its first sentence): median 0.65 s, worst 1.58 s with the speech cache (the normal case; targets: 2 s templated, 4 s overall); 2.44 s median and 4.31 s worst when the sentence has to be synthesised live. 0 of 24 spoken answers went unmatched.

Reproduce: `python bench/eval_voicebench.py --fleurs --report`, `python bench/voice_latency.py --report` (node running).

---

4. More ways in

Screenshots. The 55 ScamBench test messages rendered as phone SMS screenshots (Chromium, so Hindi is shaped as on a phone) and read back with the node's offline OCR: the same scam / not-scam decision as the typed message on 51 / 55, the identical verdict on 50 / 55; mean character error 2.2%. The app shows the read text so the person can correct it before checking. Clean renders read better than real screenshots; a set of real phone screenshots is still to be collected.

Calls. People describe calls in reported speech ("he asked for my OTP", "ओटीपी माँगा"). On 26 descriptions written by the team, 18 of 18 scam calls are flagged (digital arrest, courier, fake customer care, bank OTP, KYC, electricity, relative in trouble, lottery, UPI, loan, scheme) and 8 of 8 genuine calls are left alone. These descriptions were used to add the reported-speech phrases, so they are a tuning set, not an unbiased test; ScamBench did not change when the phrases were added.

UPI QR codes. Tests cover a shop's code (no signs, with the amount and payee spelled out), a "scan to receive cashback" code (scam), an official-sounding name on a personal UPI ID collecting a fee (scam, with a name-versus-ID warning) and a QR holding a lookalike link (scam). Every UPI card says that scanning and entering a PIN sends money and that a QR never brings money in.

---

5. Offline, privacy and safety

- **Zero egress.** A Python audit hook on the node counts every socket connection and DNS lookup the node process makes to an address outside the local network; the node page shows the count live, next to the machine's open internet connections and whether the firewall rules are active. The tests trigger the hook with a simulated outside connection and lookup and check they are counted. The firewall scripts (`scripts/firewall/`) add a block rule for every non-local address.
- **Signed packs.** Every content pack carries an Ed25519 signature; the tests alter one amount in the scheme pack and check the node refuses it, and that an unsigned pack is refused in strict mode.
- **On the phone.** After one visit to the node's HTTPS address (or to the stand-alone site, `scripts/build_tryit.py`), the phone keeps the app, its own scam check (`web/checker.js`) and benefits interview (`web/navigator.js`) and the signed packs, and answers by itself with no node and no internet; nothing leaves the phone. `bench/eval_phone_parity.py` replays 1,841 scam-check and 7,983 benefits cases through both and compares every field: 0 mismatches. `scripts/check_phone_offline.py` drives a phone browser: one visit, network off, then a scam check, a UPI QR check and a benefits interview.
- **Privacy.** The impact export is tested with messages full of names, phone numbers and UPI IDs: none appear in it; counts under 5 show as "<5"; the rupees-at-risk total is withheld until there are 5 flagged messages; the export's signature verifies. The case log refuses entries without consent, drops any name or number field, is unreadable on disk, and deletes entries older than 30 days.
- **Safety gate.** Every vetted template passes the runtime gate, and the tests check it blocks advice to share an OTP, click a link, install an app or pay a fee, catches verdict contradictions and invented amounts, and treats "never share your OTP" and "it asks you to share your OTP" as safe (`tests/test_gate.py`).

---

6. Under load and under attack

`bench/stress/run_stress.py` starts a node and plays a busy counter against it (Apple M3, 8 cores, 16 GB): 30 phones doing a whole visit at once, 300 checks in the same instant, 20 new sentences to speak at once, 10 recordings at once, and 5,000 checks in a row while the node's memory is watched.

Scenario	Request	Median	95th percentile	Slowest	Errors
session × 30	check	66 ms	338 ms	507 ms	0
session × 30	nav next	62 ms	340 ms	823 ms	0
session × 30	speech (new)	2903 ms	4714 ms	4856 ms	0
burst × 300	check	1045 ms	2179 ms	2333 ms	0
tts × 20	speech (new)	1865 ms	3282 ms	3282 ms	0
asr × 10	recognise	226 ms	271 ms	271 ms	0
soak × 5000	check	38 ms	47 ms	163 ms	0

Hostile input (360 cases: Unicode soup, malformed and 20,000-deep JSON, broken audio and images, a 144-megapixel PNG packed into 140 KB, 200 MB request bodies, path traversal, console calls without a login, PIN guessing): 0 unexpected answers, 0 files leaked, and the node kept answering. The console locks after 5 wrong PINs.

What the first run of this test found, and what changed: live speech for many phones queued behind one voice engine (now a small pool that grows only while phones wait, and one synthesis shared by every phone asking for the same sentence; a phone that waits more than 4 s reads the screen in its own voice); an empty photo crashed the QR and screenshot readers; a 0.9 MB image that unpacks to 900 MB took the node from 0.7 to 1.8 GB of memory (now refused from its header); and request bodies were read whole before their size was checked (now refused unread, by declared or streamed size).

A low-end phone (Chromium with the CPU 6× slower and a slow 3G link): first visit 6.4 s, ready to work offline after 6.4 s; offline, the first scam check took 1.04 s from tap to verdict (it starts the engine) and the next 0.88 s; a benefits result 2.44 s. Safari's engine (WebKit, as an iPhone 13) runs every flow, on the node and stand-alone (`scripts/check_phone_offline.py --browser webkit`).

---

7. Known gaps

- ScamBench v1 (600+ real messages, collected with consent) and the field morning with real users are still to come; the blind red team was written for the test, not received by real people.
- Sahayak reads Hindi, English and Hinglish. A message mostly in another script (Bengali, Tamil, Telugu, Kannada, Malayalam, Gujarati, Punjabi, Odia, Urdu, Santali, Manipuri) or in Marathi gets "could not check" and safe-default advice instead of a false "no scam signs"; reading those languages is future work.
- VoiceBench recordings, real phone screenshots, and a field test with real users and an operator are team tasks.
- Known miss on the ScamBench test split: a "pay the booking amount" government-scheme scam. The "approve the request" refund scam was fixed after the freeze (6 Oct); the frozen numbers above still count it as missed.
- The Hindi voices and the optional language model are licensed for non-commercial use only.
- HTTPS on phones needs a domain and certificate; until then voice input on phones uses the recorder-app fallback, and phones keep no offline copy from the node (the stand-alone site, on any HTTPS host, works offline after one visit).